

N-Queen Problem (9-Queen Problem)

Aim

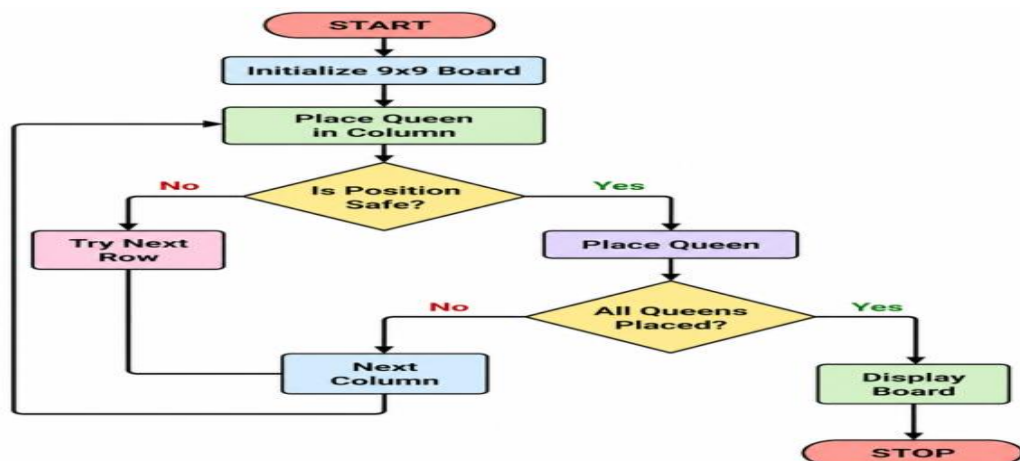
To place 9 queens on a 9×9 chessboard so that no two queens attack each other using Artificial Intelligence techniques.

Objectives

- To understand constraint satisfaction problems (CSP).
- To implement backtracking.
- To find a valid arrangement of 9 queens.
- To improve problem-solving skills in AI.

Algorithm

1. Start.
2. Place a queen in the first column.
3. Check if the position is safe.
4. If safe, place the queen and move to the next column.
5. If no safe position exists, backtrack to the previous column.
6. Repeat until all 9 queens are placed.
7. Display the board.
8. Stop.



9.

```

N = 9

def print_board(board):
    for row in board:
        print(" ".join("Q" if x else "." for x in row))

def is_safe(board, row, col):
    # Check left side
    for i in range(col):
        if board[row][i]:
            return False

    # Upper diagonal
    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j]:
            return False
        i -= 1
        j -= 1

    # Lower diagonal
    i, j = row, col
    while i < N and j >= 0:
        if board[i][j]:
            return False
        i += 1
        j -= 1

```

```

= RESTART: C:/Users/ATHIN/AppData
ssssssssss.py
Solution Found:

Q . . . . . . .
. . . . Q . . .
. Q . . . . . .
. . . . Q . . .
. . . . . . . Q
. . Q . . . . .
. . . . . . Q .
. . . Q . . . .
. . . . . Q . .

```

Result

The 9-Queen Problem was successfully solved using the **Backtracking Algorithm**. A valid arrangement of 9 queens on a 9×9 chessboard was obtained without any conflicts.